Pre-Flight Shipment Checker

Build plan for Ops Hackathon – Case 2: Conversational Shipment Booking & "Pre-Flight" Error Checker

How to use this document with Claude Code: read it top to bottom before writing any code. Section 4 says what to build in each version. When asked to build V1, build V1 fully and render the V2 tabs as visible but disabled placeholders (see 4.3). Do not build V2 features until asked.

1. Problem Statement

Booking a cross-border shipment means filling long, confusing forms. Small mistakes in them cause most of the pain later: goods held at customs, or units reported missing at the receiving warehouse. Sellers often find out days later, when their stock is already stuck.

Goal: build a solution that makes booking a shipment easy and catches mistakes before the shipment leaves.

Reference companies: Flexport (online freight forwarding), Amazon (sellers must label cartons and match declared counts exactly), Delhivery (Indian freight and cross-border logistics).

1.1 Key terms

Term	Meaning
Commercial invoice	Document listing what is in a shipment, how many, and what it is worth; customs relies on it
HS code	International code that classifies every product for customs
Declared value	Total value of the goods as stated on the paperwork
Carton / pallet	A carton is a box of units; a pallet is a platform holding many cartons
Customs hold	Customs stops a shipment until a problem with it is fixed
Delivery appointment	A booked time slot at the receiving warehouse
Transit time	How many days a shipment takes to arrive

1.2 Allowed values (test data uses only these)

Field	Allowed values
Origin country	United Kingdom
Origin city	London, Manchester, Birmingham, Leeds
Destination country	Germany, France, Netherlands
Destination city	Germany: Berlin, Frankfurt, Hamburg · France: Paris, Lyon · Netherlands: Amsterdam, Rotterdam
Currency	GBP only
Country of origin (items)	United Kingdom, China, India, Vietnam, Turkey, Germany (may be blank)

1.3 Input data (Excel template, 4 sheets)

Judges upload only Sheets 1 and 2. Sheets 3 and 4 are fixed and already filled in the template; build around them. Values marked "as the seller states it" may be wrong – checking them is our job.

Sheet 1 – Bookings (one row per shipment)

Column	Format	Example
Booking ID	ID	BK-001
Seller name	Text	Northfield Goods
Origin city · Origin country	Fixed list	Manchester · United Kingdom
Destination city · Destination country	Fixed list	Frankfurt · Germany
Destination type	Warehouse / Business address	Warehouse
Ready to ship date	DD-MM-YYYY	05-11-2026
Seller priority	Cheapest / Fastest / Most reliable / Balanced	Cheapest
Pallets (as stated)	Number	3
Cartons (as stated)	Number	90
Total declared value (as stated)	Number	5400
Currency	GBP	GBP
Total gross weight (kg) (as stated)	Number	620
Delivery appointment booked	Yes / No	Yes

Sheet 2 – Items (one row per product in a booking)

Column	Format	Example
Booking ID	ID, matches Sheet 1	BK-001
SKU · Description	Text · Text	MUG-01 · Ceramic coffee mug, 350 ml
HS code (as stated)	Number, may be blank	691200
Quantity (units) (as stated)	Number	1080
Unit price	Number, GBP	5.00
Units per carton	Number	12
Cartons (as stated)	Number	90
Unit weight (kg)	Number	0.5
Country of origin	Fixed list, may be blank	United Kingdom

Sheet 3 – Carrier options (fixed): Carrier · Service, Origin → Destination country, Price per pallet (GBP), Transit time (days), On-time rate (%), Max weight per pallet (kg). Example: Carrier A · Road economy, UK → Germany, 180, 5, 92, 500.

1.4 Sheet 4 – Rulebook (the only rules tested)

Rule	Check	Severity
R-01	Every item must have an HS code of at least 6 digits	Red
R-02	Total declared value = sum of quantity × unit price across items (within 1%)	Red
R-03	For each item, units per carton × cartons = quantity	Red
R-04	Total cartons on booking = sum of cartons across items	Red
R-05	Weight per pallet (gross weight ÷ pallets) ≤ chosen carrier's max weight per pallet	Red
R-06	No lithium batteries, aerosols or flammable liquids	Red
R-07	Total gross weight ≥ sum of unit weight × quantity across items	Amber
R-08	Shipments to a warehouse must have a delivery appointment booked	Amber
R-09	Every item must have a country of origin	Amber
R-10	Ready to ship date must not be in the past	Amber

1.5 How judges will test

Part 1 – Conversation (no file): they describe a shipment in plain language, often leaving details out. The solution must ask only for what is missing and end with a complete booking. They will change a detail mid-conversation (e.g. add a pallet) and check that the booking and its checks update.

Part 2 – File upload: they upload their own Bookings and Items sheets. Some bookings contain planted mistakes that break the rulebook.

For every booking, from either part:

- What is wrong, how serious it is, and how to fix it.
- A recommended carrier option based on the seller's priority, with the reason.
- A confirmation the seller can understand, and what happens next.
- Reasoning and clarity of explanation are judged – there is no single right answer.
- The link must work without our help and without a login.

Primer: UK government guide to creating an export (commercial) invoice – business.gov.uk › Export from UK › How to create an export invoice. It stresses correct HS codes, country of origin, weights, quantities and values.

2. Build Plan

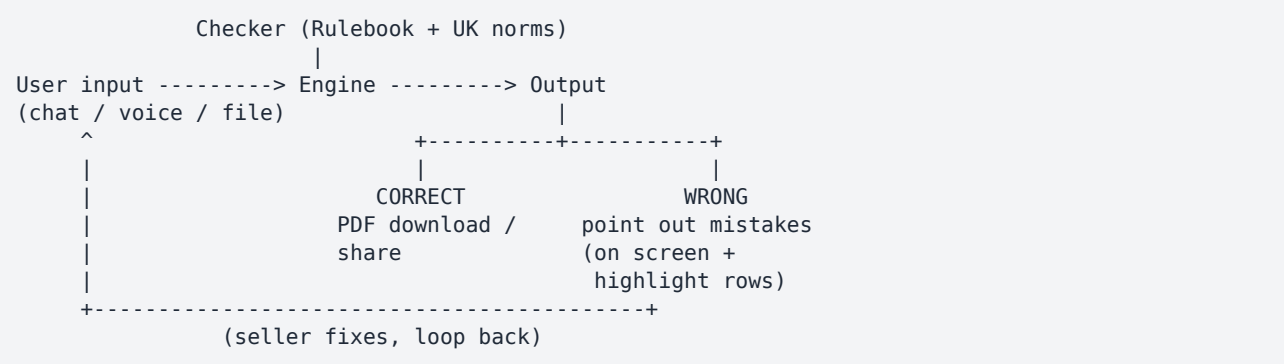
2.1 Core idea

A **pre-flight error checker**: small mistakes in form filling lead to goods held at customs. The main issue is catching them early. Combine the **Rulebook (R-01 to R-10) + UK export invoice norms** to show issues, and stress-test everything. If there is a blocking error, show it directly and do not forward the booking.

2.2 Two parts

Part	What it does
Conversation	Ask questions → take answers → validate → tell the seller what is wrong and what may be right. Handle mid-conversation changes.
File upload	Check every booking in the uploaded sheets → point out mistakes.

2.3 Overall flow



Any mid-conversation change is incorporated everywhere: the booking state, every rule check, the carrier recommendation and the confirmation all update.

2.4 Expected output per booking

- **What is wrong** – rule ID, plain-English description, the actual numbers involved.
- **Seriousness** – Red / Amber (plus Green when clean).
- **How to fix** – a concrete correction.
- **Recommended carrier option** – based on seller priority, with the reason.
- **Confirmation + what happens next.**
- **Reasoning + clarity** throughout.

2.5 Severity handling (added)

Status	Meaning	What the app does
Red	At least one Red rule broken	Booking is **blocked**. Confirm button disabled until all Red issues are fixed.
Amber	Only Amber rules broken	Booking **can be confirmed** after the seller ticks “I understand these warnings”. Warnings appear on the confirmation.
Green	No rules broken	Confirm directly.

Always show a carrier recommendation, for every booking, including blocked ones. On a Red booking label it “Recommended once fixed” so judges always see a recommendation and its reason.

2.6 Rule checks in code, Gemini for language (added)

All ten rule checks are **deterministic TypeScript functions**, not LLM calls. Planted errors are arithmetic (1% tolerance, carton sums, weight per pallet), and LLMs make arithmetic mistakes. Deterministic checks also make the reasoning auditable.

Job	Done by
R-01 to R-10 calculations, pass/fail, severity	Code (pure functions, unit tested)
Carrier filtering and ranking	Code
Turning chat/voice into booking fields	Gemini (structured JSON output)
R-06 second opinion on product descriptions	Gemini, alongside a keyword list in code
Plain-English explanations, fix wording, confirmation text	Gemini, given the code's results as input

Rule: Gemini never decides whether a rule passes. It only receives the results from code and explains them.

2.7 Carrier recommendation

- Filter carriers to the booking's route (UK → destination country).
- Check R-05 against each carrier's max weight per pallet.
- Rank by seller priority: **Cheapest** = lowest price per pallet × pallets; **Fastest** = lowest transit days; **Most reliable** = highest on-time rate; **Balanced** = best trade-off of cost, speed and reliability (normalised weighted score).
- Show the reason and the runner-up trade-off.

2.8 Steps

#	Step
1	Review with Claude
2	Rulebook – UK norms, rules + extra
3	Recommendation data → to recommend carriers
4	Build basic platform
	V1 → File upload → stress testing
	V2 → Conversation (voice + chat) → stress testing
	Testing together (V1 + V2)

3. Tech Stack

Use the latest stable release of each package at build time.

Layer	Choice	Purpose
Framework	Next.js (App Router) + TypeScript	Single app for UI and API routes; deploys natively on Vercel
UI	React + Tailwind CSS (+ shadcn/ui components)	Tabs, tables, badges, chat bubbles
Hosting	Vercel	Public link, no login required
Source control	Git + GitHub	Repo connected to Vercel for auto-deploys
Database	Neon (serverless Postgres) via @neondatabase/serverless; Drizzle ORM optional	Tables for bookings, items, carriers, rules; recommendation data
AI	Google Gemini API (@google/genai), called only from server-side API routes	Chat field extraction, R-06 description check, explanations
Excel parsing	SheetJS (xlsx)	Read uploaded Sheets 1 and 2
Validation	Zod	Booking and item schemas; validate Gemini JSON output
Dates	date-fns	Parse DD-MM-YYYY, R-10, estimated delivery date
PDF export	jsPDF + jspdf-autotable (or @react-pdf/renderer)	Downloadable confirmation / issue report
Voice (V2)	Browser Web Speech API (speech-to-text + speech synthesis)	Voice tab; transcript feeds the same chat engine
Testing	Vitest	Unit tests for every rule and the recommender

3.1 Environment variables (Vercel)

```
GEMINI_API_KEY=...        # server-side only, never exposed to the browser
DATABASE_URL=...          # Neon connection string
```

3.2 Database (Neon)

Four tables mirroring the four sheets: **bookings**, **items**, **carriers** (Sheet 3, seeded), **rules** (Sheet 4, seeded). The recommendation engine reads **carriers**.

3.3 Suggested project structure

```
/app
  page.tsx                # tabs: File Upload | Conversation (Chat, Voice)
  /api/check/route.ts     # run rules + recommendation on a booking
  /api/upload/route.ts    # parse Excel -> bookings[]
  /api/chat/route.ts      # V2: Gemini extraction + reply
/lib
  /rules/r01.ts ... r10.ts # one pure function per rule
  /rules/index.ts         # runAllRules(booking, carrier)
  recommend.ts            # filter + rank carriers by priority
  parseExcel.ts           # SheetJS -> typed Booking/Item objects
  schema.ts               # Zod types: Booking, Item, Carrier, Issue
  explain.ts              # Gemini: results -> plain English
/db
  schema.ts, seed.ts      # Neon tables; seed carriers + rules
/components
  FileUploadTab.tsx, ChatTab.tsx, VoiceTab.tsx,
  BookingCard.tsx, IssueList.tsx, CarrierCard.tsx, Confirmation.tsx
/tests
  rules.test.ts, recommend.test.ts
```

4. Versions – V1 and V2

4.1 UI layout (same in both versions)

```
+-----------------------------------------------------------+
|  Pre-Flight Shipment Checker                              |
+-----------------------------------------------------------+
|  [ File Upload ]   [ Conversation ]                       |  <- main tabs
+-----------------------------------------------------------+
|  Conversation tab only:                                   |
|      [ Chat ]   [ Voice ]                                 |  <- sub-tabs
+-----------------------------------------------------------+
```

Both main tabs and both Conversation sub-tabs are **always rendered**, in every version.

4.2 V1 – File Upload

Scope

- File Upload tab fully working.
- Upload .xlsx with Sheets 1 (Bookings) and 2 (Items); parse with SheetJS.
- Load Sheets 3 and 4 from the database (seeded from the template).
- Run R-01 to R-10 per booking in code (2.6); apply severity handling (2.5).
- Recommend a carrier per booking based on seller priority, with the reason (2.7).
- Show results on screen: summary of all bookings (Red / Amber / Green counts), then one card per booking.

Each booking card shows

- Status badge (Red / Amber / Green).
- Issues list: rule ID, severity, what is wrong (with numbers), how to fix.
- Highlighted rows in the bookings and items tables where the problem is.
- Recommended carrier + reason (“Recommended once fixed” if Red).
- Confirmation in plain English + what happens next.
- Confirm button (disabled if Red; tick-box acknowledgement if Amber).
- PDF download / share of the confirmation or issue report.

Stress testing (V1)

- Build a test workbook with a booking that breaks each rule R-01 to R-10.
- Add bookings that break several rules at once, and fully clean bookings.
- Unit-test each rule function and the recommender with Vitest.
- Open the deployed Vercel link in an incognito window to confirm no login is needed.

4.3 V2 placeholders while building V1 (required)

When building V1, the V2 parts must be **visible but non-functional**:

- The **Conversation** main tab is shown, with its **Chat** and **Voice** sub-tabs.
- They are greyed out with a “Coming in V2” badge.
- Clicking them does nothing beyond showing a short message such as “Conversation booking arrives in V2.” No chat input, no microphone, no API calls.
- Do not remove or hide these tabs at any point in V1.

Instruction for Claude Code: when asked to build V1, build the full File Upload tab and render the Conversation tab with Chat and Voice sub-tabs as disabled placeholders. Do not implement any V2 logic.

4.4 V2 – Conversation (Chat + Voice)

Chat sub-tab

- Seller describes the shipment in plain language; Gemini extracts fields into the booking state (JSON, validated with Zod).
- Ask only for what is missing; validate each answer.
- Tell the seller what is wrong and what may be right, using the same rule engine as V1.
- Mid-conversation changes (e.g. “add a pallet”) update the booking state, re-run all checks and the carrier recommendation, and show what changed.
- A live booking summary panel beside the chat shows current fields, status and issues.
- Ends with the same booking card, recommendation, confirmation and PDF as V1.

Voice sub-tab

- Microphone button → speech-to-text (Web Speech API) → same chat engine as the Chat sub-tab.
- Replies are shown as text and read aloud (speech synthesis).
- Same live booking summary panel and outputs as Chat.
- If the browser does not support speech, show a clear message and suggest the Chat sub-tab.

Stress testing (V2)

- Vague seller who leaves out most details.
- Seller who changes a detail mid-way (add a pallet, change destination).
- Seller who contradicts themselves.
- Same scenarios via Voice.

4.5 Testing together

- Same booking entered via file and via chat must give identical rule results and recommendation.
- All tabs work on the public Vercel link with no login.
- Final run through both judge test parts end to end.

4.6 Version summary

	File Upload tab	Conversation – Chat	Conversation – Voice
V1	Fully working	Visible, disabled, “Coming in V2”	Visible, disabled, “Coming in V2”
V2	Fully working	Fully working	Fully working